



Yenepoya (Deemed University)

Project – High Level Design

on

AI-Powered Student Chatbot using Python & String Matching

**SUBMITTED BY:
SHIYAS MOHAMMED
23BCAICD100
26052**

Industry Mentor: Sumit Kumar Shukla

Guided by: Sindhu

Table of Contents

1. Introduction	3
1.1. Scope of the Document	3
1.2. Intended Audience	3
1.3. System Overview	4
2. System Design.	5
2.1. Application Design	5
2.2. Process Flow.....	6
2.3. Information Flow	7
2.4. Components Design	7
2.5. Responses	8
2.6. API Catalogue	9
3. Data Design.	10
3.1. Data Model	10
3.2. Data Flow.....	11
3.3. Data Retention and Migration	11
4. Interfaces	12
5. State and Session Management	12
6. Caching	13
7. Non-Functional Requirements	13
7.1. Security Aspects	13
7.2. Performance Aspects	14
8. References	14

HLD - Cloud Cost Intelligence Platform for Finance

1. Introduction

TheEdTechAssistant is an academic project developed under the IBM Industry Mentorship Programme at JAIN (Deemed-to-be University). It is a web-based conversational chatbot built entirely in Python using the standard library, without any external frameworks or packages. The system allows registered students to log in, ask questions about courses, exams, attendance, and other campus-related topics, and receive instant keyword-matched responses. This High Level Design (HLD) document provides a comprehensive architectural view of the system. It covers system objectives, technology stack, architectural diagrams described in text, component responsibilities, data flows, API contracts, session management strategy, security posture, nonfunctional requirements, constraints, assumptions, and known limitations. This document is intended to be read alongside the Low Level Design (LLD) and the Final Project Report.

1.1. Scope of the Document

This HLD document covers the following areas:

- System architecture and technology decisions
- Component design and responsibilities
- All HTTP API endpoints (contracts, request/response schemas)
- Data structures, data flow, and in-memory data model
- Session and state management design
- Frontend architecture and JavaScript interaction patterns
- Security design and known limitations
- Non-functional requirements (performance, scalability, maintainability)
- Assumptions, constraints, and future enhancement roadmap

1.2. Intended Audience

Audience	Purpose
Student Developer (Shiyas)	Primary implementer; uses this as design reference
Faculty Advisor	Academic evaluation and guidance
IBM Industry Mentor	Industry review and feedback
Project Evaluators	Assessment of design quality and completeness
Future Developers	Onboarding and system extension

1.3. System Overview

The EdTech Assistant follows a Single-Tier Monolithic Architecture. The entire application — web server, business logic, session store, chatbot engine, and frontend — is contained within a single Python file (edtech.py). There are no external service calls, no database connections, and no framework dependencies.

Layer	Technology	Role
Presentation	HTML5 / CSS3 / Vanilla JS	Renders login UI, chat window, quick-action chips; communicates via fetch() API
Application / Logic	Python 3 (http.server)	HTTP routing, authentication, session management, keyword matching
Data / State	Python dicts in RAM	Stores session objects, user credentials, chat history, response knowledge base

2. System Design

2.1. Application Design

The application is structured around a central HTTP Handler class with helper functions. The following diagram (described textually) shows the relationships between all components:

Component	Type	Interacts With
ThreadingHTTPServer	Server bootstrap	OS TCP socket; Handler class
Handler (do_GET)	HTTP method handler	get_session(); sessions{}; HTML string; _set_headers(); _send_json()
Handler (do_POST)	HTTP method	get_session(); users{}; sessions{}; responses[];
	handler	_read_json(); _send_json()
get_session()	Session manager	sessions{}; http.cookies; secrets; handler.new_sid
_set_headers()	Response helper	HTTP response stream; Set-Cookie header; Content-Type
_read_json()	Request parser	Content-Length header; request body; json.loads()
_send_json()	Response helper	_set_headers(); json.dumps(); HTTP response body
responses[]	Knowledge base	Iterated by do_POST /chat; read-only
users{}	Credential store	Read by do_POST /login; read-only
sessions{}	In-memory store	Written by get_session(); read/written by do_GET and do_POST
HTML string	Embedded frontend	Served by do_GET /; contains all JS, CSS, HTML

2.2. Process Flow

The chat flow handles five distinct message types before falling back to keyword matching:

Message Type	Trigger Keyword	Server Action
Auth check	(any message, not logged in)	Return HTTP 401 { requiresLogin: true }
Empty input	(empty string after strip)	Return HTTP 400 { response: 'Please enter a message.' }
Clear history	clear	Reset session['history'] = []; return { historyCleared: true }
View history	history	Return { showHistory: true, history: session['history'] }
Logout	exit	Set session['user'] = None, clear history; return { ended: true }
Keyword match	any other input	Scan responses[]; return first match or fallback; append to history

2.3. Information Flow

The following describes the complete information flow for the most common use case (a student sending a chat message):

Step	From	To	Data Transferred
1	Student	Browser JS	User types query and presses Enter or clicks Send
2	Browser JS	Python Server	HTTP POST /chat, Cookie: sid=<token>, Body: { message: 'exam' }
3	Python Server	sessions{}	Lookup sessions[sid] → returns { user: 'shiyas', history: [...] }
4	Python Server	responses[]	Iterate: check if 'exam' or 'test' in 'exam' → match found
5	Python Server	sessions{}	Append 'You: exam' and 'Bot [HH:MM:SS]: Exams start from
6	Python Server	Browser JS	HTTP 200, Body: { response: 'Exams start...', timestamp:
7	Browser JS	Chat Display	Append user bubble and bot bubble to DOM; scroll to bottom

2.4. Components Design

ThreadingHTTPServer

The server is instantiated at the entry point with HOST='127.0.0.1' and PORT=5001.

ThreadingHTTPServer (a subclass of TCPServer) creates a new daemon thread for each incoming connection, allowing multiple browser tabs or simultaneous requests to be handled without blocking.

```
server = ThreadingHTTPServer((HOST, PORT), Handler) webbrowser.open(f'http://{HOST}:{PORT}')
server.serve_forever()
```

ChartEngine — renders a pie chart (service cost share with percentage labels) and a bar chart (monthly spend trend) using Matplotlib's Pyplot interface.

Control Loop (Notebook) — the Jupyter Notebook cell execution sequence orchestrates the full pipeline:
 Load → Clean → Transform → Aggregate → Visualise.

2.5.responses[] — Knowledge Base

#	Keywords	Response Text
1	help	You can ask: status, course, exam, attendance, history, clear, exit
2	status, system	All EdTech systems are running normally.
3	course, subject	Courses: AI, Data Science, Cloud Computing.
4	exam, test	Exams start from 10th April.
5	attendance, present	Your attendance is 85%.
6	hi, hello	Hello! How can I assist you?
—	(no match)	Sorry, I didn't understand that.

2.6 API Catalogue

TheEdTechAssistant exposes four HTTP endpoints. All POST endpoints accept and return JSON. All endpoints are served on <http://127.0.0.1:5001>.

Attribute	Value
Method	GET
Path	/
Authentication	None required
Request Headers	Cookie: sid=<token> (optional; new session created if absent)
Response Status	HTTP 200
Response Content-Type	text/html; charset=utf-8
Response Body	Full HTML page (embedded Python string HTML)
Side Effects	May set Set-Cookie: sid=<new_token>; Path=/; HttpOnly if new session

3. Data Design

3.1 Data Model

All application state is stored in two module-level Python dictionaries: `users{}` (read-only credential store) and `sessions{}` (read-write session store). No file system or database is used.

Data Flow — Session Lifecycle

Lifecycle Stage	State Change
First visit (GET /)	New sid generated; <code>sessions[sid] = { user: None, login_attempts: 0, history: [] }</code> ; Set-Cookie sent
Successful login	<code>sessions[sid]['user'] = username</code> ; login_attempts reset to 0; history reset to []
Failed login (< 3)	<code>sessions[sid]['login_attempts'] += 1</code>
Failed login (= 3)	<code>sessions[sid]['login_attempts'] = 3</code> ; session locked
Chat message sent	<code>sessions[sid]['history'].append('You: <msg>')</code> ; <code>.append('Bot [HH:MM:SS]: <reply>')</code>
Clear command	<code>sessions[sid]['history'] = []</code>
Logout (exit)	<code>sessions[sid]['user'] = None</code> ; <code>sessions[sid]['history'] = []</code>
Server shutdown	All sessions lost; no persistence mechanism

Data Retention and Migration

Data Retention: No data is persisted beyond the server process lifetime. Chat history exists only in RAM and is cleared on server restart or on user logout.

Data Migration: Not applicable for this version. A future version using SQLite would require a schema migration strategy. The proposed schema for a persistent version would be:

```
-- Future SQLite schema
CREATE TABLE sessions (sid TEXT PRIMARY KEY, username TEXT, created_at INTEGER,
last_active INTEGER);
CREATE TABLE history (id INTEGER PRIMARY KEY, sid TEXT, role TEXT, message TEXT,
timestamp TEXT);
```

responses[] — Knowledge Base

```
responses: list[tuple[list[str], str]] = [
    (['help'], 'You can ask: status, course, exam, attendance, history, clear,
exit'),
    (['status', 'system'], 'All EdTech systems are running normally.'),
    (['course', 'subject'], 'Courses: AI, Data Science, Cloud Computing.'),
    (['exam', 'test'], 'Exams start from 10th April.'),
```

4. Interfaces

The frontend UI is a single-page application (SPA-like) served as static HTML from the Python server. It transitions between three states without page reloads:

UI State	Description
Logged Out	Login panel visible; chat section hidden (class: hidden); error message area for failed logins
Logged In	Login panel hidden; chat section visible; user's name displayed in topbar; history loaded from GET /session
Locked	Both panels show lockout message; input disabled; prompt to restart server

5. State and Session Management

Session Token Design

Session tokens are generated using Python's secrets module, which provides cryptographically strong random bytes suitable for security-sensitive applications. The token is 16 bytes (128 bits) of random data, encoded as a 32-character hexadecimal string:

```
sid = secrets.token_hex(16) # e.g., 'a3f8d2c1b4e9f7a0d5c2e8b1f3a6d9c4'
```

128 bits of entropy makes the token computationally infeasible to guess by brute force. The token is issued as an HttpOnly cookie, preventing JavaScript from reading it directly and reducing XSS exposure.

6. Caching

No explicit HTTP caching is configured. The server does not send Cache-Control, ETag, or Last-Modified headers. Every GET / request serves a fresh copy of the HTML string from memory. Since the HTML is a compile-time constant (Python string), the cost is negligible — it is simply a memory read and a socket write.

The sessions{} dictionary functions as an implicit application-level cache: user state and conversation history are cached in RAM for the duration of the session, avoiding the need to reconstruct state on every request.

7. Non-Functional Requirements

7.1. Security Aspects

Security Requirement	Implementation	Gap / Limitation
Authentication	Username + password checked against users{}	No MFA; plaintext passwords
Session management	128-bit random token in HttpOnly cookie	No expiry; no server-side invalidation beyond logout
Brute-force protection	3-attempt lockout per session	Lockout is per-session, not per-IP; new session bypasses it
Transport security	HTTP on localhost only	No TLS; acceptable for academic; unacceptable for production
Input sanitisation	strip() and lower() applied to inputs	No HTML escaping; XSS risk if output were rendered as HTML (it is text-only)
CSRF protection	Not implemented	POST endpoints accept any origin; mitigate with SameSite=Strict + CSRF token
Injection attacks	No SQL/shell execution	No injection surface beyond JSON parsing; safe by design

7.2. Performance Aspects

Aspect	Detail
Concurrency model	One OS thread per HTTP request (ThreadingHTTPServer). Suitable for < 50 simultaneous users.
Response time	time.sleep(0.5) introduces 500ms artificial delay for UX; actual processing is < 1ms
Keyword matching	$O(n * k)$ linear scan where $n=6$ tuples, $k=\text{avg } 2$ keywords. Completes in microseconds.
Memory usage	Approx. 10–20MB Python process. Each session adds ~1KB. Grows linearly with sessions.

8. References

Python 3 Official Documentation — http.server module: <https://docs.python.org/3/library/http.server.html>
 Python 3 Official Documentation — secrets module: <https://docs.python.org/3/library/secrets.html>
 Python 3 Official Documentation — http.cookies module: <https://docs.python.org/3/library/http.cookies.html>
 Python 3 Official Documentation — json module: <https://docs.python.org/3/library/json.html>
 Mozilla Developer Network — HTTP Cookie: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>
 Mozilla Developer Network — Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
 OWASP — Session Management Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
 IBM EdTech Project Guidelines — Academic Year 2025-26
 JAIN (Deemed-to-be University) Project Submission Manual